Cliniface 7.0 Windows Build Process (Draft)

Disclaimer and License

Cliniface is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version. Cliniface is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see http://www.gnu.org/licenses/.

The Cliniface repository can be found at:

https://github.com/frontiersi/Cliniface

Directory setup

Your home directory will depend on your operating system. If you are on linux, you can easily get to your home directory by opening a terminal and typing "**cd ~**". If you want to see the full location of your home directoy on Linux, you can then type "**pwd**". On Windows, your home directory should be stored under "**c:\Users**YourUsername"

Within your home directory, please create the following directory structure, where Home is a stand in for your home directory location:

HOME\local_source

Home\local_libs

Home\local_build

Home\dev

Home\dev\libs

Home\pylibs

MS Visual Studio Build Tools

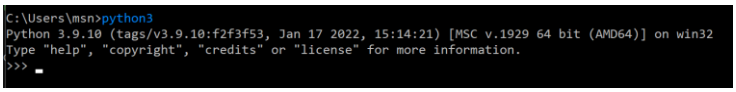
You will need the MS Visual Studio Build Tools 2019, specifically version 16.11 or later. You do not need Visual Studio installed to install the tools, and if you have Visual Studio installed already you may not have the tool version required by Cliniface. Microsoft change their website periodically, so it is beyond

the scope of this guide to provide you with up to date instructions on how to find and install the Build Tools, please use your preferred search engine to look for information.

NOTE: Any of the 16.x versions should work, but there is a known issue with 16.7 which will throw a trait_type error later due to a backend change to how inheritance is managed, this error is definitely fixed in 16.11

Python

You will require Python for the Cliniface Install. If you do not know if you have Python already, you can run a command line prompt (Windows + R > cmd), then type "Python3" and hit enter. If Python is installed it will provide a small information prompt telling you the version such as:



You can see from this prompt that this computer is running Python 3.9.10. As long as you are running Python 3.5 or higher, it should be fine. If you do not have Python installed, use your preferred search engine to locate and install an appropriate version.

Qt5

You will need to acquire Qt5, which can be installed by going to https://www.qt.io/. On the website at the time of writing this guide, you want to click "**Browse Qt Tools**", scroll to the bottom of the page and click "**Get your free Qt trial**", then scroll to near the bottom of the page and click "Go open source". Scroll to near the bottom of the Open source page and click "**Download the Qt Online Installer**", then "**Download**".

This will download the Qt Online installer, which when complete we will need to run to install several tools we need. Once downloaded, run the executable and you will be asked to sign into your Qt account. If you don't already have an account, follow the link provided in the installer to sign up, don't worry, it's free!

Once you've signed in, you will need to read and accept some agreements and click next a few times until you get to the Installation Folder page. Specify **C:\Qt** as the install directory, and select **Custom Installation**. You can install it elsewhere if you want, but instructions later in this guide will assume this location so if you install it elsewhere you will need to apply those changes yourself to later instructions.

You will need the following components and ONLY the following components for the Cliniface installation:

Qt 5.15.2 > MSVC 2019 64-bit, Qt Charts, and Qt Data Visulisation only.

Developer and Designer Tools > Qt Installer Framework, CMake, Ninja, and OpenSSL 64-bit binaries.

Commented [RP1]: I think it's enough to say that the MSVC build tools are needed, and the particular version. Honestly, MS keeps changing where the tools are downloaded from every time I look for them. Details about version problems (like you've written) are important though.

Commented [SP2R1]: Roger. Will reduce to the key points.

Commented [RP3]: Python 3.5 and higher is fine (in fact, that's the highest supported version of Python officially supported by Ubuntu Xenial, so the scripts have to work with that version. Mostly this means that the scripts don't use f-strings which were introduced in version 3.6).



Finish the install process and Qt will download which may take awhile. While it's installing, you can move on and install other required software.

Git

We will be using Git to clone several code repositories, so you will need to install it. Goto https://git-scm.com/download/win and download an appropriate installer. When in doubt, go for 64bit as that's what any modern computer should be running.

Git LFS

Some of the files in the Cliniface libraries are larger than the default git tool allows, so Git LFS is required.

https://git-lfs.github.com/

Commented [RP4]: You don't need the MinGW compilers and you only need MSVC 2019 64-bit. You also only need OpenSSL 64-bit binaries and don't need the source code.

Commented [SP5R4]: Corrected. Bit of overkill crept in during the build process, thanks.

Setup Command prompt

To build all the components for Cliniface correctly, you will need to call them from a Command Prompt that is setup to run with the Visual Studio tools and QT5. To do this, open the Command Prompt (Win + R > cmd). Right click the Command prompt in your taskbar and pin it to the taskbar.

Then right click the pinned Command Prompt icon, hover over the Command prompt option and right click it, then click properties. In properties, paste the following text into the Target box:

```
%windir%\system32\cmd.exe /k %Qt5%\bin\qtenv2.bat & "C:\Program Files (x86)\Microsoft Visual Studio\2019\Community\VC\Auxiliary\Build\vcvarsall.bat" amd64
```

Hit apply, ok, and close the command prompt properties. If you now click the Command prompt in your taskbar, it should tell you it is running with Qt and Visual studio when it opens, like this:



If you do not get information that Visual Studio Developer Command Prompt is running, double check you have setup the taskbar shortcut correctly.

Required libraries

In order to build Cliniface, you will need the following libraries/binaries. Some of the libraries, such as OpenCV, reference other libraries, such as VTK, so they should be built in the order below:

VTK 9.1.0
AssImp 5.1.6
Boost 1.73
OpenCV 4.3 with OpenCV_contrib 4.3.0
Eigen 3
Lua 5 with Sol2 header

Commented [RP6]: You need the contrib libraries too which are a separate source download that you need to match version numbers with and then specify the location of these modules when configuring the OpenCV build with CMake.

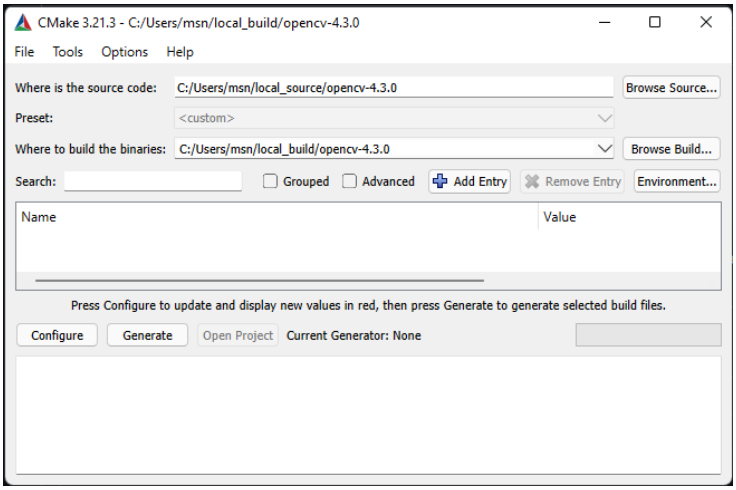
Commented [RP7]: Need the third-party Sol2 header for Lua which allows C++ and Lua to play nice.

Nanoflann
Quazip

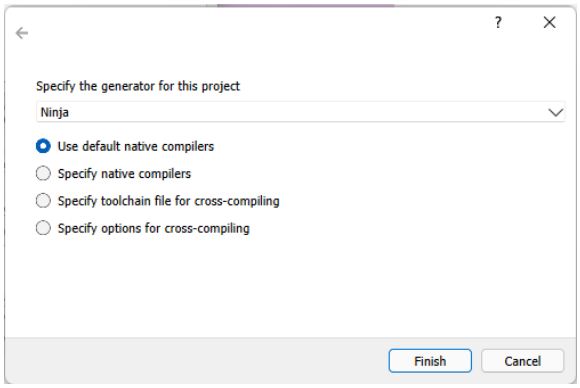
Which should all be placed in "**Home\local_libs**" so the various packages can find the components they need. Note: AssImp need to be placed in **HOME\local_libs\AssImp**, specifically.

These packages can be difficult to install, and it's not in the scope of this guide to provide instructions for installing each of them. We will provide some instructions below, but search engines, forums, and your own ingenuity will have to help you through the rest.

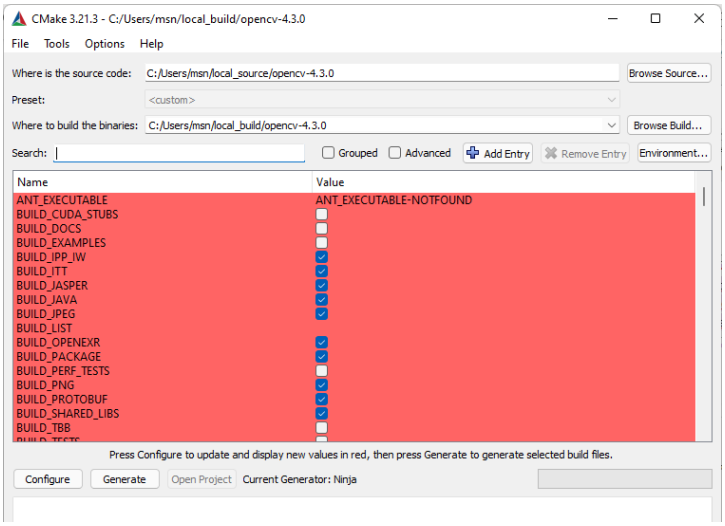
As an example, we'll look at the process for building OpenCV 4.3.0. You should be able to acquire the source code from www.github.com/opencv/, which you should then unzip into Home\local_source. You will also want to acquire OpenCV_contrib 4.3.0, and unzip this into your Home\local_source\ as well. If you then open your Home\local_source directory in a command prompt, and use 'cd opencv-4.3.0' to change to the directory then run 'cmake-gui .' which will present you with a window similar to the following:



You will need to tell the CMake gui where to find the source, as well as where to build the file to (Home\local_build\opencv-4.3.0). If you then click the 'Configure' button, the interface will present you with a prompt asking what tools you want to use. You should specify 'Ninja' for the compiler:



After you hit Finish the top pane of the Cmake-gui should fill with text that is highlighted in red. The red highlight indicates that this is a new setting, which will help you identify when a change to one setting makes a new setting available. We need to alter several of these settings, with the easiest way to find them be using the search bar.



Set the following values:

BUILD_opencv_world	ON
WITH_QT	OFF
BUILD_opencv_python_bindings_generator	ON
BUILD_opencv_python_tests	ON
BUILD_CUDA_STUBS	OFF, except following

Formatted Table

through to BUILD_ZLIB	
BUILD_IPP_IW	ON
BUILD_ITT	ON
BUILD_JAVA	ON
BUILD_PACKAGE	ON
BUILD_PROTOBUF	ON
BUILD_SHARED_LIBS	ON
BUILD_PERF_TESTS	OFF
BUILD_TESTS	OFF
OPENCV_EXTRA_MODULES_PATH	HOME/local_source/opencv_contrib-4.x/modules
CMAKE_INSTALL_PREFIX	HOME/local_lib/opencv-4.3.0/

NOTE: Even on Windows, when entering paths in CMAKE you will need to use '/'

After the values are set, hit **Configure** and the list will refresh. Check you're happy with all the values, then press **Generate**. This will create the build file in HOME\local_build\opencv-4.3.0, open a Command Prompt and move to this directory, then use the command:

```
Cmake --build .
```

Cmake will then use the build file to build all the necessary files. When it's done, check the output to make sure it completed successfully, otherwise you will need to try and work through any issues raised. Once it does complete successfully, you will then need to run the install command:

```
Cmake --install .
```

Cmake will again output as it goes, and the process will take awhile for opencv. Again when it's done, check the output to ensure it completed successfully. If it didn't, you will need to troubleshoot. If it did, great! Move onto the next one.

Github Clone

Next you will need to clone the various repositories from Github into the required directories. You can do this via several means, but we will describe how to do this from the command prompt (Win + R > cmd) using Git.

You will need to acquire:

libbuild	https://github.com/richeytastic/libbuild
rlib	https://github.com/richeytastic/rlib
rimg	https://github.com/richeytastic/rimg
r3d	https://github.com/richeytastic/r3d
r3dio	https://github.com/richeytastic/r3dio

Formatted Table

Commented [RP8]: Building and installing third-party library dependencies is actually the biggest headache of the entire build process, so I'd like the details documented please with the caveat that other users may encounter other issues, but that we have encountered the following yadda yadda.

Commented [RP9R8]: Also need Quazip (which is included as a separate tgz in the QTools repo)

Commented [SP10R8]: Roger. You copied a lot of the libraries to me via thumb drive, so I don't have notes on all of them, but will add detail. =)

r3dvis	https://github.com/richeytastic/r3dvis
rNonRigid	https://github.com/richeytastic/rNonRigid
QTools	https://github.com/richeytastic/QTools
FaceTools	https://github.com/frontiersi/facetools
Cliniface	https://github.com/frontiersi/Cliniface

To do this from your command prompt, navigate to **Home\dev\libs** and use the following command:

```
git clone https://github.com/richeytastic/libbuild.git
```

This will begin the process of cloning the libbuild components into Home\dev\libs\libbuild. Once it is done, repeat the process for rlib through to FaceTools:

```
git clone https://github.com/richeytastic/rlib.git
git clone https://github.com/richeytastic/rimg.git
…
git clone https://github.com/richeytastic/qtools.git
git clone https://github.com/richeytastic/facetools.git
```

Note: The "**…**" above is a stand in for the other git clone commands.

Once all the supporting components have been cloned from Github, move up a directory to **Home\dev** and then clone Cliniface

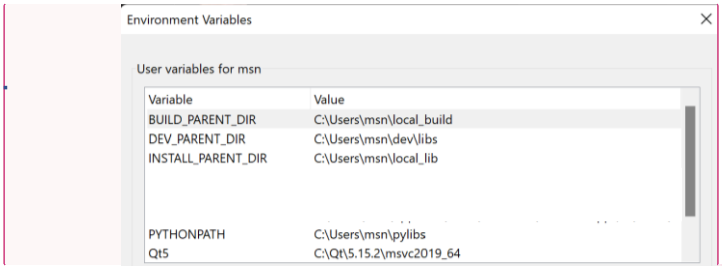
```
git clone https://github.com/frontiersi/Cliniface.git
```

Environment Setup

Cliniface will need a few environment variables set in the Registry in order to build and run correctly. To edit the registry, open a File Explorer window, then right click "This PC" and select Properties. In the window that opens, select **System > About > Advanced System Settings > Environment Variables**.

Note: We encountered an issue where, for some reason, when we attempted to set the Environment Variables by using the Windows Search to open directly to the Environment Variable screen, they did not save. If you do not follow the process above, make sure you check that the environment variables have been saved.

We will want to create the following User Variables by clicking "New..." then enter the Variable name and Value, replacing msn with your username:



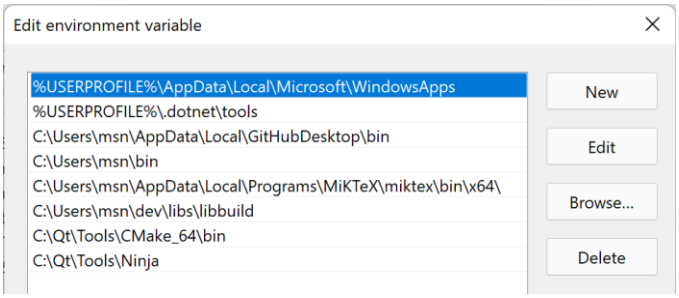
Commented [RP11]: Missing image

Commented [SP12R11]: Showing for me. Odd. Will investigate.

As also shown in this table:

BUILD_PARENT_DIR	Home\local_build
DEV_PARENT_DIR	Home\dev\libs
INSTALL_PARENT_DIR	Home\local_lib
PYTHONPATH	Home\pylibs
Qt5	C:\Qt\5.15.2\msvc2019_64

Then click the “Path” variable in the Environment Variables list and select edit, you should be presented with a box similar to this:



You will need to add two new variables to your path so the build process knows where to find CMake_64 and Ninja. If you installed QT to your C drive, the path for this should be “C:\Qt\Tools\CMake_64\bin” and “C:\Qt\Tools\Ninja” respectively. You will also need to add a path to your libbuild directory, “Home\dev\libs\libbuild”

Commented [RP13]: You also need to set the PYTHONPATH env var to be %USERPROFILE%\pylibs and ensure there's a soft link in that folder to libbuilds\CMakeBuild.py

Also need to set Qt5 to point to the Qt5 directory for the installed version of MSVC.

Once you’ve added all the path variables, click “Ok” on all the opened windows to save the changes until you get back to the system properties screen.

Build components

To run the components correctly you will need to set up a symbolic link for the makelibs.py file located in **Home\dev\libs\libbuilds**, to do this you will need to run your command prompt as Administrator (Right click taskbar shortcut >Right click Command Prompt > Run as Administrator).

Commented [SP14R13]: These were shown in the image that didn't load for you for some reason. Hm. Might write out to make explicit.

Then from the elevated command prompt, navigate to **Home\pylibs** and run the command:

run mklink makelibs.py ..\dev\libs\libbuild\makelibs.py

We are now ready to try and build all the components!

You should close the elevated Command Prompt and open a new one with regular permissions. Within this prompt if you type the command:

makelibs

You should get the following output:



If not, there's an issue. Go back and check that you setup the Environment variables correctly, and named all the directories consistently.

You can now type:

makelibs all install

And the libbuild component will run it's scripts and attempt to build all the components. This will take awhile and produce a lot of output. You may prefer to specify a single library, such as rlib, and build one at a time. If so, you should do so in the order they are listed in the command prompt documentation, as later libraries such as r3dio make use of earlier libraries such as r3d.

Either way, check the output generated by the build script for any errors. Some warnings will be created, such as "loss of accuracy", which can be ignored, we are only concerned with any FAILED messages that indicate an error that prevent compilation. If such an error has occurred, you can use:

```
makelibs all clean
```

This command will remove all the currently compiled builds so you can recompile them all or one at a time to identify the issue preventing completion. Boost is commonly a source of problems, as it's a complex library. You only need the headers and binaries, which can be downloaded pre-compiled from the Boost.org, just make sure you get the correct version (1.73, 64bit).

The error messages produced should indicate to you what library failed and why, and you will need to google and work to resolve the issue, then re-build that library and any that rely upon it.

Build Cliniface

Once all the libraries have been built on your system, you can attempt to build Cliniface itself. If everything else has compiled without error, this process will most likely go smoothly.

Before you build Cliniface, you need to set up its directory. Like before, you will need to run a Command Prompt as Administrator in order to create the symbolic link.

In **Home\local_builds**, run the command:

```
mkdir Cliniface
```

Finally, change directory to **Home\dev\Cliniface** and run the command "**makeme**":

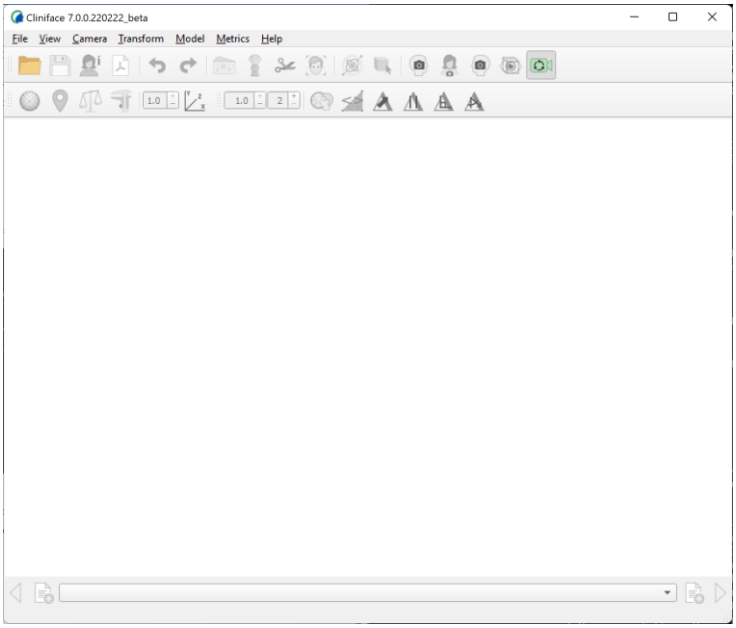


Then run "**makeme release**", and the Cliniface build scripts will go through and build the Cliniface executable. The build process shouldn't take too long, once done check for any error messages. If everything finished installing without error, you are now ready to run Cliniface!

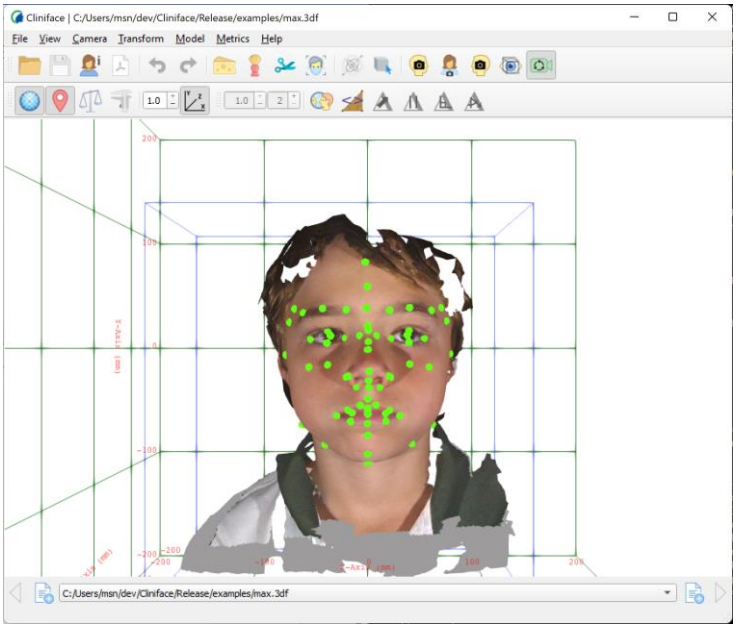
From **Home\dev\Cliniface\Release\bin**, run the command:

```
.\cliniface
```

And Cliniface should load and present you with it's GUI:



You can load a sample dataset to test the functionality from **Home\dev\Cliniface\Release\examples**, such as the max.3df dataset:


Cliniface | C:/Users/msn/dev/Cliniface/Release/examples/max.3df
File View Camera Transform Model Metrics Help
C:/Users/msn/dev/Cliniface/Release/examples/max.3df